

プログラミングI 演習

Computer Programming I: Exercises

第14回 復習

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

本日の講義

本日の目標

- これまでの内容(重要なキーワード含む)を確認しよう

本日の資料

1. 前回のおさらい(第13回)
2. これまでの内容確認(第1回～第13回)

2. 前回のおさらい

構造体とは

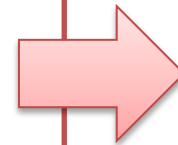
情報の中には、関連性の強い
グループのようなものがある

例:プロフィール(氏名、年齢、性別、血液型・・・)
日時(年、月、日、曜日、時、分、秒)
住所録(氏名、住所、電話番号・・・)



関連性の高いものは、まとまっていた
方が扱いやすい

変数をグループにまとめる機能が
あれば、プログラムが楽に(美しく)
なるのではないかな？



構造体

というものがある

2. 前回のおさらい

構造体

```
typedef struct  
{  
    int num;  
    int den;  
} Bunsuu;
```

```
int main(void)  
{  
    Bunsuu X; //分数X  
    Bunsuu Y; //分数Y  
    Bunsuu ans; //答え  
  
    //分数のかけ算  
    ans.num = X.num * Y.num;  
    ans.den = X.den * Y.den;  
    return 0;  
}
```

構造体(struct)の型定義

Bunsuuという新しい型(intとかfloatとかの仲間)を作っている

プログラマが自由に変数を構造化して、新しい型を作ることができる

numやden(構造体の中の変数)をメンバとよぶ

2. 前回のおさらい

構造体

```
typedef struct
{
    int num;
    int den;
} Bunsuu;
```

```
int main(void)
{
```

```
    Bunsuu X; //分数X
    Bunsuu Y; //分数Y
    Bunsuu ans; //答え
```

```
    //分数のかけ算
    ans.num = X.num * Y.num;
    ans.den = X.den * Y.den;
    return 0;
```

```
}
```

構造体の変数(実体)の宣言

int型やfloat型と同様に、
「Bunsuu型」の変数(X,Y,ans)
を作成している

2. 前回のおさらい

構造体

```
typedef struct
{
    int num;
    int den;
} Bunsuu;

int main(void)
{
    Bunsuu X; //分数X
    Bunsuu Y; //分数Y
    Bunsuu ans; //答え

    //分数のかけ算
    ans.num = X.num * Y.num;
    ans.den = X.den * Y.den;
    return 0;
}
```

構造体メンバの参照方法

変数名.メンバ名

というように記述する
(〇〇の(≡「.」)△ △ という感じ)

分数Xの分母=10ならば..
X.den = 10;

2. 前回のおさらい

演習13-2 5科目×5人分の成績処理

5人分の国語, 数学, 理科, 社会, 英語(各100点満点)の点数を入力し, 各人の合計点・平均点, 各教科の平均点を出力するプログラムを作成せよ. ただし, 以下の構造体を利用すること. (変数名は変更しても良い)

```
typedef struct{
    int Jpn; /* 国語 */
    int Mat; /* 数学 */
    int Sci; /* 理科 */
    int Soc; /* 社会 */
    int Eng; /* 英語 */
} Student;
```

2. 前回のおさらい

演習13-2 5科目×5人分の成績処理

ヒント: 構造体の定義 + 値の入出力 (まずはこのとおり実行してみよう)

```
#include<stdio.h>
#include<stdlib.h>
typedef struct{
    int Jpn; int Mat; int Sci; int Soc; int Eng;
}Student;

int main(void){
    int i,j;
    Student s[5];

    /* 値の入力 */
    for(i=0; i<5; i++){
        s[i].Jpn = rand()%101; s[i].Mat = rand()%101; s[i].Sci = rand()%101;
        s[i].Soc = rand()%101; s[i].Eng = rand()%101;
    }
    /* 値の表示 */
    printf("学生 国語 数学 理科 社会 英語\n");
    for(i=0; i<5; i++){
        printf("s[%d]", i);
        printf("%5d", s[i].Jpn); printf("%5d", s[i].Mat);
        printf("%5d", s[i].Sci); printf("%5d", s[i].Soc);
        printf("%5d\n", s[i].Eng);
    }
    return 0;
}
```


2. 前回のおさらい

演習13-2 解答例(1/3): 各種定義と値の入力

```
#include<stdio.h>
#include<stdlib.h>
#define STUDENT_NUM 5
typedef struct{
    int Jpn; int Mat; int Sci; int Soc; int Eng;
}Student;
```

構造体定義→問題のとおり

```
int main(void){
    int i,sum;
    Student s[STUDENT_NUM];
```

Student型 配列の定義(人数分)

```
/* 値の入力 */
for(i=0; i<STUDENT_NUM; i++){
    s[i].Jpn = rand()%101;
    s[i].Mat = rand()%101;
    s[i].Sci = rand()%101;
    s[i].Soc = rand()%101;
    s[i].Eng = rand()%101;
}
```

乱数で点数(0~100)を入力
(キー入力・ファイル入力でも良い)

2. 前回のおさらい

演習13-2 解答例(2/3): 値の出力と各人の合計・平均

```
/* 値の表示＋各人の合計点・平均点 */
printf(" 学生 国語 数学 理科 社会 英語 合計 平均¥n");
for(i=0; i<STUDENT_NUM; i++){
    printf("s[%2d]", i);      sum = 0;
    printf("%5d", s[i].Jpn);  sum += s[i].Jpn;
    printf("%5d", s[i].Mat);  sum += s[i].Mat;
    printf("%5d", s[i].Sci);  sum += s[i].Sci;
    printf("%5d", s[i].Soc);  sum += s[i].Soc;
    printf("%5d", s[i].Eng);  sum += s[i].Eng;
    printf("%5d", sum);
    printf("%5.1f¥n", (double)sum/5);
}
```

最後に各i(各学生)の平均点を
出力する(小数の扱いに注意)

各i(各学生)について、5教科の
合計得点を求める

2. 前回のおさらい

演習13-2 解答例(3/3): 各教科の平均点出力

学生を表す「i」を変えれば、教科ごとの合計・平均を求めることができる

```
/* 各教科の平均点 */
printf(" 平均");
sum = 0; for(i=0; i<STUDENT_NUM; i++) sum += s[i].Jpn;
printf("%5.1f", (double)sum/STUDENT_NUM); /* 国語 */

sum = 0; for(i=0; i<STUDENT_NUM; i++) sum += s[i].Mat;
printf("%5.1f", (double)sum/STUDENT_NUM); /* 数学 */

sum = 0; for(i=0; i<STUDENT_NUM; i++) sum += s[i].Sci;
printf("%5.1f", (double)sum/STUDENT_NUM); /* 理科 */

sum = 0; for(i=0; i<STUDENT_NUM; i++) sum += s[i].Soc;
printf("%5.1f", (double)sum/STUDENT_NUM); /* 社会 */

sum = 0; for(i=0; i<STUDENT_NUM; i++) sum += s[i].Eng;
printf("%5.1f¥n", (double)sum/STUDENT_NUM); /* 英語 */

return 0;

}
```

2. 前回のおさらい

演習13-2 実行結果

学生	国語	数学	理科	社会	英語	合計	平均
s[0]	29	74	80	41	47	271	54.2
s[1]	21	52	21	18	15	127	25.4
s[2]	14	36	53	9	47	159	31.8
s[3]	59	57	41	100	51	308	61.6
s[4]	30	19	47	5	37	138	27.6
平均	30.6	47.6	48.4	34.6	39.4		

2. 前回のおさらい

発展13-1 30人分の成績処理＋ファイル入出力

補足資料「Result.txt」に30人分の漢字氏名, フリガナ, 各教科の得点が記入されている. 各教科の平均点, 各人の合計・平均・偏差値を出力できるプログラムを作成せよ. (出力先はコンソール上でもファイルでも良い)

ファイルの形式

氏名01¥tフリガナ01¥t国語01¥t数学01¥t理科01¥t社会01¥t英語01
氏名02¥tフリガナ02¥t国語02¥t数学02¥t理科02¥t社会02¥t英語02
....
氏名30¥tフリガナ30¥t国語30¥t数学30¥t理科30¥t社会30¥t英語30

30人分ある

サンプル「Result.txt」の点数は架空のものです.
実在の人物とは無関係です.

2. 前回のおさらい

発展13-1 解答例の一部: 各種定義とファイルからの読み込み

```
#include<stdio.h>
#include<math.h>
#define STUDENT_NUM 30
```

```
/* 構造体定義 */
```

```
typedef struct{
    char Name[32]; char Kana[32];
    int Jpn; int Mat; int Sci; int Soc; int Eng;
    int Sum; /* 合計得点 */
}Student;
```

演習13-2のプログラムを活かすため
教科ごとに変数名をつけるように変更

偏差値を求めるための、標準偏差の計算で利用

```
/* ファイルから読み込み → 値の入力 */
```

```
void ReadFile(Student StudentList[]){
    int ID;
    FILE *fp = fopen("Result.txt", "r");
    for(ID = 0; ID < STUDENT_NUM; ID++){
        fscanf(fp, "%s¥t%s¥t%d¥t%d¥t%d¥t%d¥t%d", StudentList[ID].Name,
            StudentList[ID].Kana, &StudentList[ID].Jpn,
            &StudentList[ID].Mat, &StudentList[ID].Sci,
            &StudentList[ID].Soc, &StudentList[ID].Eng);
    }
    fclose(fp);
}
```

形式を指定した読み込み

1. これまでの内容確認 (第1回)

第1回の目標

- 各自のPCで, C言語プログラミングができる環境を整える
- printf文を使って, (プログラムに書いた)文字をそのまま出力できるようにする
- 改行文字を覚える

キーワード

- printf関数 (printf文)
- main関数
- 改行文字「`\n`」 (バックスラッシュ+`n`)

1. これまでの内容確認 (第2回)

第2回の目標

- 変数と変数の型を覚える
- (数学とは異なる) 代入文を覚える
- 変数を利用した演算ができるようになる
- scanf文を使って変数への入力, printf文を使って変数の内容
を出力することができる

キーワード

- printf文と「%d」「%f」「%lf」「%c」
- scanf文と「%d」「%f」「%lf」「%c」
- 変数の型「int」「float」「double」「char」
- 演算子「+」「-」「*」「/」「%」「++」「--」「=(代入)」
- 出力時の整形「%3d」「%10.6lf」「%.10lf」

1. これまでの内容確認 (第3回)

第3回の目標

- 条件分岐 (if文, if～else文) ができるようになる
- 簡単な流れ図 (フローチャート) が読めるようになる
- 関係演算子と論理演算子を覚える

キーワード

- if文, if～else文
- 流れ図 (フローチャート)
- 関係演算子「==」「!=」「<」「>」「<=」「>=」
- 論理演算子「&&」「||」

1. これまでの内容確認(第4回)

第4回の目標

- ループ文(for文, while文)を覚える
- 簡単な流れ図(フローチャート)が読めるようになる(再掲)
- 簡単な数値計算ができるようになる

キーワード

- for文
- while文

1. これまでの内容確認 (第5回)

第5回の目標

- (一次元)配列の利用方法を覚える
- 配列を宣言できる
- #define の利用方法を覚える
- 配列を用いた簡単な演算ができる

キーワード

- 配列とその宣言 → `int a[10];`
- 配列の初期化 → `int a[10] = {0,1,2,3,4,5,6,7,8,9};`
 `int a[10] = {}; /*すべて0で初期化*/`
- 配列の要素 → 0から始まる
- #define [シンボル名][文字列]
 → [シンボル名] を[文字列]に置換

1. これまでの内容確認 (第6回)

第6回の目標

- 関数の役割を知ろう
- プロトタイプ宣言を知ろう
- ヘッダファイルの役割を知ろう
- 関数を使った簡単なプログラムを作成しよう

キーワード

- 関数
- 引数(ひきすう)
- 戻り値, return文
- プロトタイプ宣言
- ヘッダファイルと呼び出し「`#include<stdio.h>`」「`<stdlib.h>`」
- void型:「空である」ということを表す特殊な型

1. これまでの内容確認 (第7回)

第7回の目標

- ポインタとその利点について知ろう
- ポインタの使い方を覚えよう
- ポインタを使った簡単なプログラムを作成・実行してみよう

キーワード

- 値渡しとポインタ渡し
- ポインタ型変数 (宣言の違い: `int *a` と `int a`)
- ポインタ型変数における「*」や「&」の利用
例) `int a=5; int *pA=&a;`

1. これまでの内容確認(第8回)

第8回の目標

- 関数の役割を知ろう(再掲)
- 関数の引数として配列を渡せるようになろう

キーワード

- ポインタ渡し
- 関数に渡すのは配列名(同名のポインタ)

1. これまでの内容確認 (第9回)

第9回の目標

- ポインタと配列の関係を覚えよう
- 配列をポインタで表現してみよう

本日の資料

- 配列の宣言とポインタの宣言 (配列宣言 `a[ ]` → `*a`)
- 配列名とポインタの関係 (配列名 `a` → `&a[0]`)
- 配列とメモリ確保 → 配列はメモリ上に連続して置かれる
- ポインタ型変数の足し算

1. これまでの内容確認 (第10回)

第10回の目標

- コンピュータ内部での文字の表現方法を知ろう
- 文字の入力や出力ができるようになるろう
- 文字や文字列を用いたプログラムを書けるようになるろう

キーワード

- 文字コード
- '(シングルクォテーション) → 1バイト文字の表現
- 終端文字 ('\0')
- "(ダブルクォテーション) → 文字列表現
- %s → 文字列の入出力形式
- strlen関数, strcpy関数, strcmp関数, strcat関数

1. これまでの内容確認(第11回)

第11回の目標

- ファイルから値を入力できるようになろう
- プログラムの実行結果をファイルに出力できるようになろう

キーワード

- ファイルポインタ(FILE *fp)
- fopen関数, fscanf関数, fprintf関数, fclose関数
- "r", "w", "a" → ファイルを開くときのモード

1. これまでの内容確認 (第12回)

第12回の目標

- 乱数を発生させるプログラムを作ろう
- 「擬似乱数」について知ろう
- 乱数を用いた簡単なシミュレーションを行ってみよう

キーワード

- 擬似乱数 (擬似乱数列)
- rand()関数
- srand()関数

1. これまでの内容確認 (第13回)

第13回の目標

- 構造体の利点を知ろう
- 構造体の宣言方法を覚えよう
- 構造体を利用したプログラムを作成してみよう

キーワード

- 構造体の型宣言 例) `typedef struct { .... } Bunsuu;`
- 変数(実体)の宣言 例) `Bunsuu X;`
- 構造体メンバの参照 変数名.メンバ名
- 構造体のコピー `X = Y;`

3. 美しいプログラム作成のために

プログラムの美しさ

アルゴリズム

後期以降の講義・演習等で身に付ける
処理対象の本質をとらえた考え方をする

見た目

デバッグのしやすさが向上
他人との共同作業では必須

3. 美しいプログラム作成のために

インデント(字下げ)～before～

インデントが無いと・・・

*/*得点調整関数*/*

```
void Adjust(int Score[100][5], double AddPoint[5]){  
int i,j  
for(j = 0; j < 100; j++){  
for(i = 0; i < 5; i++) Score[j][i] = Score[j][i] + AddPoint[i];}}
```

ループが2つ並んでいる？

3. 美しいプログラム作成のために

インデント(字下げ) ～after～

関数やループ、条件分岐等のブロックごとに
字下げを行うことで見やすさが向上

```
/*得点調整関数*/  
void Adjust(int Score[100][5], double AddPoint[5])  
{  
    int i,j;                /*ループ用変数*/  
    for(j = 0; j < 100; j++) /*人数分のループ*/  
    {  
        for(i = 0; i < 5; i++) /*科目用のループ*/  
        {  
            Score[j][i] = Score[j][i] + AddPoint[i];  
        }  
    }  
}
```

インデントをすることで、2重ループであることや、
それらの始まりと終わりが一目でわかる。

3. 美しいプログラム作成のために

コメント～before～

関数の使い方や、処理の意味が書かれていないと・・・

```
//-----  
void DrawPlaneTriangles()  
{  
    glBegin(GL_TRIANGLES);  
    for(unsigned int TID = 0; TID<TriangleData.size();TID++)  
    {  
        for(int VID=0;VID<3;VID++)  
        {  
            VECTOR4 &Pos  = VertexData[ TriangleData[TID].VertexID[VID] ].Position;  
            RGB &Color     = VertexData[ TriangleData[TID].VertexID[VID] ].Color;  
            glColor3f( Color.Red, Color.Green, Color.Blue);  
            glVertex3f( Pos.x, Pos.y, Pos.z);  
        }  
    }  
    glEnd();  
}
```

何をする関数かわからない

3. 美しいプログラム作成のために

コメント～after～

関数の使い方や、処理の説明をコメントで書くことで、**関数の理解が深まる**

```
//-----  
// 3角形の描画(座標変換なし)  
void DrawPlaneTriangles()  
{  
    //図形の描画を開始(三角形)  
    glBegin(GL_TRIANGLES);  
    //全ての3角形に対して処理する  
    for(unsigned int TID = 0; TID<TriangleData.size();TID++)  
    {  
        //3角形3つの頂点をOpnGLに転送  
        for(int VID=0;VID<3;VID++)  
        {  
            //長いので、位置と色への参照を別変数に格納  
            //頂点データを現在の3角形 の 頂点番号から見つけてその位置情報  
            VECTOR4 &Pos  = VertexData[ TriangleData[TID].VertexID[VID] ].Position;  
            //頂点データを現在の3角形 の 頂点番号から見つけてその色情報  
            RGB &Color    = VertexData[ TriangleData[TID].VertexID[VID] ].Color;  
            //頂点の色の転送  
            glColor3f( Color.Red, Color.Green, Color.Blue);  
            //頂点位置の転送  
            glVertex3f( Pos.x, Pos.y, Pos.z);  
        }  
    }  
    //図形の描画を終了  
    glEnd();  
}
```

個々の関数や変数の詳細は分からないが、
この関数が何をしているかを理解できる

予告：後期プログラミングII演習の実施内容

1年後期プログラミングII演習

第1回	Linux上でのCプログラムの実行	第8回	ポインタと文字列
第2回	Cプログラムの基礎	第9回	ポインタ配列を用いたソート
第3回	再帰関数を用いたプログラム	第10回	構造体
第4回	配列を用いたソート	第11回	単方向リスト
第5回	文字列のソート	第12回	双方向リスト
第6回	ポインタ	第13回	双方向リストを用いたソート
第7回	ポインタと動的メモリ確保	第14回	木構造

第1回～第2回 復習

第3回～第5回 再帰・ソーティング(数値 & 文字列)

第6回～第9回 ポインタ + ソーティング

第10回～第14回 構造体 + ソーティング・リスト

第6回以降はポインタを頻繁に使います。
後期までにマスターしよう！

プログラミング I 演習の資料は以上です.